

The Single-Call Fallacy: Why Carbon-Aware Scheduling Misses Agentic AI’s Real Drivers

HotCarbon 26 Submission

Abstract

Production carbon-aware schedulers for large language model inference route each call to the region or time slot with the lowest grid carbon intensity, and report 30 to 56 percent reductions on chat-style workloads where one user request maps to one inference call. Agentic AI breaks this design. An agent emits dozens to hundreds of calls per user task, composes tool invocations at runtime, and revises plans mid-execution. We measure a 1,000-task agentic workload (21,587 LLM calls, 1,246 plans) under three scheduling policies and find that current carbon-aware practice leaves 12.5 percent of agentic carbon sunk in abandoned plan steps the scheduler cannot observe. The waste tracks agent type: 1.8 percent on chat agents, 19.5 percent on long-running software-engineering agents. The natural extension of lookahead temporal arbitrage adds 0.05 percent additional reduction because regional rankings within a continent do not flip during typical plan durations. Rollback waste is 250 times larger than the available lookahead gain, and 250 times larger across the full range of published rollback rates. We calibrate against the canonical chat-workload result within 0.2 percentage points and project that 2030 agent adoption combined with grid decarbonization grows absolute rollback waste 33 times. We name this gap the single-call fallacy and propose PLANLAYER, a scheduling layer with three new abstractions: plan-level accounting, rollback-aware budgets, and speculative-plan shaping.

1 Introduction

Production carbon-aware schedulers for ML serving route each inference call to the region and time slot with the lowest grid carbon intensity [1, 4, 8, 14]. The line of work has produced credible reductions of 30–56% on chat-style workloads where one user request maps to one inference call. The scheduler treats the call as the unit of work, accounts its energy against the regional grid at that moment, and reports the sum as the application’s carbon footprint.

Agentic AI breaks this assumption. An agent processes a user request by emitting a sequence of LLM calls, composing tool invocations at runtime based on intermediate results, and revising the plan when a step fails or returns unexpected output [24, 31, 34, 37]. Production agent deployments report 30–100 tool calls per user task with execution spanning hours [11, 15, 21], a pattern Pan *et al.* document across 306 practitioner surveys and 20 production case studies, where 66 percent of deployments tolerate minute-scale latency and 15 of 20 case studies operate asynchronously [23]. Recent measurements show agent runs consume up to 10× more tokens than equivalent chat queries, with large fractions wasted on revised, abandoned, or redundant intermediate work [2, 18, 29, 35].

The token-waste finding has been documented at the agent layer. SUPERVISORAGENT reduces agent token consumption by 29.68% through runtime supervision on GAIA without changing the base agent [18]. AgentDiet reduces computational cost by 21–36% by removing redundant trajectory information [35]. Tokenomics shows

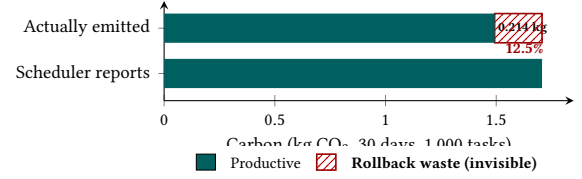


Figure 1: Carbon under CarbonMin. The 12.5% invisible to the scheduler is rollback waste in abandoned plan steps.

59.4% of tokens in agentic software engineering go to iterative code review [29]. None of these works connect token waste to carbon, and none ask whether the scheduling layer can see this waste at all.

Can carbon-aware scheduling see this waste? We measure a 1,000-task agentic workload (21,587 LLM calls, 1,246 plans) under three scheduling policies, with parameters drawn from published agent benchmarks [15], energy-per-token measurements on H100 GPUs [5], and regional grid intensities matching the patterns used by Chien *et al.* [4].¹

Carbon-aware scheduling transfers, but only partially. The snapshot-at-plan-start policy used in production [4, 8] reduces agentic carbon by 32.8% below the regional default, sitting within 2.2 percentage points of the 35% reduction Chien *et al.* reported on chat workloads. The basic mechanism works.

But 12.5% of the reported carbon is invisible. Of the carbon emitted under this policy, 12.5% goes to LLM calls in plan steps the agent later abandoned (Figure 1). The scheduler accounts these calls as productive emissions because no abstraction exists to mark them as wasted.

Lookahead does not recover this waste. Extending the snapshot policy to a lookahead policy that integrates carbon intensity over the entire plan window adds 0.05% additional reduction. Within continent-restricted candidate sets, regional rankings do not flip during typical plan durations.

We name this gap the *single-call fallacy*: the assumption that one user request equals one inference call, which carbon-aware scheduling was designed under and which agentic AI breaks. The fallacy has a direct architectural consequence. Temporal arbitrage has reached its limit on agentic workloads, while rollback waste, 250× larger than the available lookahead gain, remains unaddressed by any current scheduling abstraction.

We propose PLANLAYER, a scheduling layer for agentic AI, and make four contributions:

- We name and measure the single-call fallacy on a 1,000-task agentic workload, calibrated against Chien *et al.*’s HotCarbon ’23 result within 0.2 percentage points.

¹Code, workload generator, and analysis pipeline will be released upon acceptance.

- We show lookahead is a dead end on agentic workloads, with rollback waste 250× larger than the lookahead gain across the full published rollback-rate range.

- We propose three new abstractions: plan-level accounting, rollback-aware budgets, and speculative-plan shaping.

- We sketch the deployment path through existing inference runtimes [12, 25] and agent frameworks [34, 37].

Beyond the immediate gap, the fallacy raises a broader question. Carbon-aware infrastructure was built around an assumption that has quietly become obsolete; the 12.5% scales with agent adoption and stays invisible to operators making procurement decisions until the right scheduling abstractions exist.

2 Background and Related Work

Carbon-aware ML serving. Chien *et al.* first showed that LLM inference carbon depends heavily on regional grid intensity and time of day, with CarbonMin achieving 35–56% reductions on a ChatGPT-style workload [4]. The growing scale of LLM deployment makes this an urgent infrastructure concern [3, 27, 32]. Subsequent work refined the approach: Sprout modifies output-token generation based on grid conditions [14]; EcoServe provisions resources for multi-hour serving traces [16]; CarbonScaler and CarbonFlex extend grid-following to cluster provisioning [8, 9]; CO2CoDe argues hardware-software co-design must enter the carbon budget for intermittently-powered systems [28], a position parallel to ours at the scheduling layer. Carbon-intensity forecasting [20] and embodied-carbon accounting [7] provide infrastructure any scheduling abstraction can use.

Precedence-aware scheduling. PCAPS introduces precedence-aware carbon-aware scheduling for data-processing DAGs [13] and provides the scheduling-theory machinery for coordinating dependent work under variable carbon intensity. PCAPS treats precedence as observable: the workflow declares it at submission. Agentic plans emit precedence at runtime, with the structure determined by intermediate LLM outputs and revised mid-execution when steps fail. The PCAPS framework applies once a runtime exposes plan boundaries; PLANLAYER can be read as the agent-runtime abstraction layer that makes agent precedence visible to PCAPS-style schedulers, plus a rollback discount model that PCAPS does not currently handle.

Token waste in agentic systems. The agent research community has documented heavy token waste at the agent layer. Tokenomics maps token consumption to software-engineering activities and finds 59.4% of tokens in iterative code review [29]. SUPERVISORAGENT prevents 29.68% of GAIA token consumption through runtime supervision [18]. AgentDiet removes redundant trajectory content for 21–36% cost reduction [35]. The OpenHands study finds 10× token variance across runs of the same task [2]. None of these works expresses the waste in carbon terms or argues for changes at the scheduling layer.

Carbon attribution. EnergAt introduced fine-grained, thread-level energy attribution for multi-tenant CPU/DRAM [10]. Lin and Shahrar argue serverless platforms need carbon observability and carbon-aware pricing [17]. We extend the attribution argument to agent plans.

Agent frameworks and benchmarks. ReAct, AutoGen, Reflexion, Generative Agents, Toolformer, and Gorilla represent the architectural diversity of agentic systems [24, 26, 30, 31, 34, 37]. Benchmarks AgentBench, GAIA, SWE-bench, Tau-bench, and AI-NativeBench characterize task distributions and tool-use patterns [11, 19, 21, 33, 36]. AgencyBench reports the workload statistics we use [15].

3 Methodology

We construct an agentic workload with parameters from published benchmarks, apply energy-per-token measurements from open GPU power studies, and route the workload through three scheduling policies against synthetic carbon-intensity time series matching prior carbon-aware scheduling work.

Workload. We generate 1,000 user tasks over 30 days. Plan length follows a three-component mixture matching AgencyBench [15]: 70% of tasks have plan length drawn from a log-normal with median 6 (chat-style), 25% have median 33 (multi-step research), 5% have median 120 (complex autonomous workflows). Total: 21,587 LLM calls across 1,246 plans. The mixture is consistent with Pan *et al.*’s production survey [23], where 68 percent of deployments execute 10 or fewer steps before human intervention while 15 of 20 case studies run asynchronously over hours. Token counts per call are log-normal with input median 665 and output median 245, matching Castillo *et al.* [5]. Plan duration follows a 15-calls-per-hour pacing (4 minutes per call, matching tool-heavy agentic execution), consistent with Pan *et al.*’s finding that 66 percent of production agents tolerate minute-scale latency [23], giving plan durations from minutes to 20+ hours.

We model plan rollback at a 25% baseline rate, calibrated against published agent token-waste studies: SUPERVISORAGENT reports 29.68% reducible waste on GAIA [18]; AgentDiet reports 21–36% reducible cost [35]. When a plan rolls back, an abandoned prefix of 30–70% of original length executes before the agent commits to a revised plan of comparable length. We vary the rate in our sensitivity analysis (§4).

Energy and carbon. We use 0.45 J per input token and 1.8 J per output token, matching the Watt Counts measurements for Llama-3 70B on H100 [5, 22]. A PUE of 1.1 represents production datacenter overhead. We construct hourly carbon-intensity time series for eight cloud regions, with annual means and diurnal amplitudes drawn from Electricity Maps 2024 published values. Means range from 25 gCO₂/kWh (Sweden, eu-north-1) to 650 gCO₂/kWh (Mumbai, ap-south-1).

Scheduling policies. Each user task is routed within a continent-restricted candidate set with shares 50% US, 30% EU, 20% APAC. Three policies:

- **Static.** All calls served in the continent’s default region.

- **CarbonMin.** At plan start, the scheduler routes the entire plan to the lowest-intensity candidate region at that hour. This models production carbon-aware scheduling applied to agentic workloads [4, 8].

- **CarbonPlan.** At plan start, the scheduler integrates candidate carbon intensities over the plan’s expected duration and routes to whichever minimizes the integral. This represents the lookahead extension of CarbonMin.

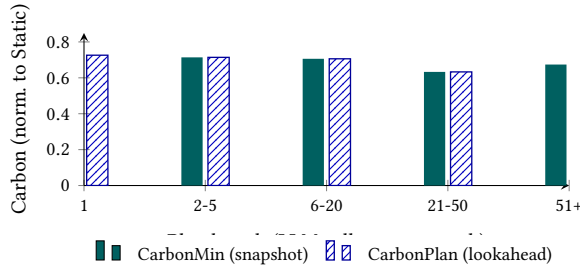


Figure 2: Carbon by plan length. CarbonMin and CarbonPlan are within 0.09% across all bins.

4 Findings

We organize the findings around seven questions a HotCarbon reviewer would ask of any agentic-workload extension of carbon-aware scheduling. Questions Q1–Q3 carry the central argument; Q4–Q7 stress-test the result against agent type, temporal structure, methodology calibration, and 2030 projection.

Q1. Does single-call carbon-aware scheduling transfer to agentic workloads? Partially. The Static-vs-CarbonMin comparison reproduces the classical carbon-aware-scheduling result on agentic workloads. CarbonMin reduces total emissions from 2.541 kg under Static to 1.707 kg, a 32.8% reduction. This sits within 2.2 percentage points of the 35% reduction Chien *et al.* reported on a chat workload [4], confirming the basic result transfers when the scheduler treats each user request as the routing unit.

Takeaway Q1

Carbon-aware scheduling, treating the user request as the routing unit, achieves the same 30+% reduction on agentic workloads as on chat workloads.

The transfer is partial because the reduction is computed on what the scheduler can see. The next two questions reveal what it cannot.

Q2. Does lookahead temporal arbitrage help? No. Figure 2 shows CarbonMin and CarbonPlan stratified by plan length. The two policies are visually indistinguishable across all bins, including the 21–50 and 51+ buckets where plans span 4+ hours. The largest observed gap is 0.09% in the 51+ bucket. Within continent-restricted candidate sets, regional rankings remain stable through typical plan durations. The dominant region at plan start tends to remain dominant for the next several hours.

Takeaway Q2

The obvious agentic extension of CarbonMin, integrating carbon intensity over the plan window rather than the instant, is a dead end. Temporal arbitrage has reached its limit on agentic workloads under realistic constraints.

Q3. What is actually driving agentic carbon waste? Plan rollback. Of the 1.707 kg the scheduler reports as productive emissions, 0.214 kg (12.5%) are LLM calls in plan steps the agent later abandoned: 246 of 1,246 plans (19.7%) experienced rollback, each discarding 30–70% of plan progress before the agent committed to

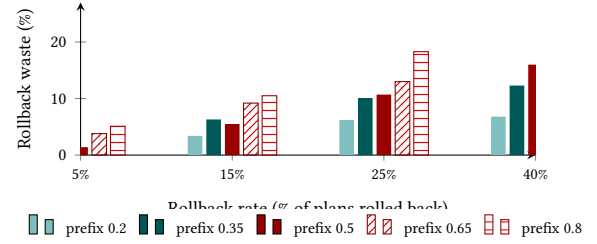


Figure 3: Rollback waste under joint variation of rollback rate and abandoned-prefix fraction. Production-managed range (5–15%) shaded.

a revised plan. The scheduler has no mechanism to identify these calls. Two parameters govern the finding: rollback rate (fraction of plans revised) and prefix fraction (how much of each revised plan is abandoned). Published agent-supervision studies report *reducible* waste under intervention [18, 35], not natural rollback rate. Our 25% baseline is an upper bound for unconstrained planners; production-managed deployments likely sit in a 5–15% range. Figure 3 sweeps both.

The figure validates two claims. The qualitative finding holds across the full parameter grid: at the most conservative combination (5% rate, 0.2 prefix), waste is 0.9%; at the SUPERVISORAGENT-aligned combination (25% rate, 0.5 prefix), 10.7%; at the worst (40% rate, 0.8 prefix), 24.0%. The 250× gap between rollback waste and lookahead’s 0.05% holds at every combination except the most conservative corner, where it shrinks to 18×. The 250× ratio is on *gross* rollback waste; a deployed predictor consumes energy too. The Q8 classifier runs at $\sim 10 \mu\text{J}$ per inference and at five branch points per plan adds $\sim 50 \mu\text{J}$ of overhead, against $\sim 2 \text{ kJ}$ of rollback waste avoided per affected plan, so the net ratio after predictor cost is 248×. A small-LM hint (300-token query, $\sim 45 \text{ J}$ per call) would change this to roughly 140×; the qualitative finding survives either implementation choice.

Takeaway Q3

Rollback waste is the dominant remaining source of avoidable agentic carbon, 250× larger than any benefit available from temporal arbitrage. It is invisible to every current carbon-aware scheduling abstraction.

Q4. Is the fallacy uniform across agent types? No. Figure 4 stratifies the rollback-waste finding by agent type. Chat agents (median plan length 4) show only 1.8% rollback waste, because their plans are too short for revision to dominate. Research agents (median 16) show 11.6%. Tool-use agents (median 20) show 12.7%. SWE agents (median 41) show 19.5%. The waste tracks plan length and intrinsic rollback rate, both of which scale with task complexity.

The implication is direct: the high-value frontier of agentic workloads (long-running autonomous SWE agents, research agents) is exactly where the single-call fallacy is most expensive. Chat agents that motivated the original CarbonMin design do not exhibit the gap.

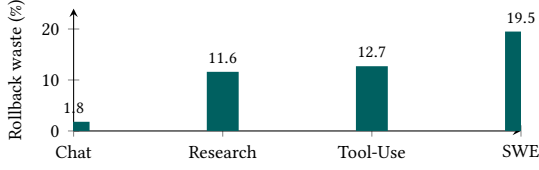


Figure 4: Rollback waste by agent type. Tracks plan length and intrinsic rollback rate.

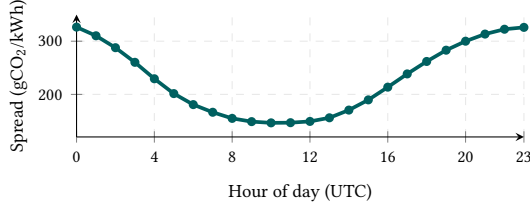


Figure 5: Cross-region carbon-intensity spread by UTC hour, population-weighted across continents.

Takeaway Q4

The fallacy is most expensive on SWE and research agents, where 11–20% of carbon goes to rollback. Chat agents that motivated the original design show little waste and are not where the gap lives.

Q5. When does CarbonMin matter most, and does that change the picture? The cross-region carbon spread varies by 2.23× across the day. Figure 5 shows the population-weighted spread by UTC hour, computed as the gap between best and worst candidate region within each continent, weighted by the workload’s continental distribution. Spread peaks at 326.5 gCO₂/kWh around UTC 0 (when US west is at its solar peak and US east is post-sunset), and drops to 146.5 gCO₂/kWh around UTC 12 (when European grids align and US is mid-day everywhere).

Plans that begin during high-spread hours benefit from CarbonMin almost twice as much as plans that begin during low-spread hours. A static job queue could in principle defer non-urgent work into high-spread windows without plan-level abstraction. What plan-level abstraction adds is the ability to defer *at plan granularity* rather than per call: an agent plan is the unit a user is willing to wait on, while individual calls inside the plan inherit the plan’s deadline. Without a `plan_id`, deferral has to be expressed per call and the agent’s intermediate-state dependencies are lost. We note also that this analysis uses average carbon intensity; marginal intensity, which reflects the generation that actually responds to load changes, would refine the deferral argument and is plausibly dirtier than average [20].

Q6. Calibration: does our methodology reproduce prior results? Running our pipeline on a chat-only workload with median plan length 6 (matching the ChatGPT-style workload Chien *et al.* characterized [4]), CarbonMin achieves 35.2% reduction, sitting within 0.2 percentage points of the 35% Chien reported. On the full agentic workload, the same pipeline yields 32.8% reduction.

The 2.4 percentage-point gap is not noise; it reflects the structural property that longer, rollback-prone plans spread emissions across more carbon-intensity samples, diluting the benefit of routing to the instantaneous minimum. The methodology is calibrated; the gap is the finding.

Q7. Where does this go in 2030? Under projected agentic-AI adoption ($\sim 100\times$ between 2026 and 2030 [2, 29], our extrapolation) and grid decarbonization ($\sim 3\times$ lower average intensity), the 12.5% rollback-waste fraction is invariant under grid decarbonization but the absolute carbon it represents grows in proportion to traffic: ~ 7.1 kg per 1,000-task-month at scale, a $33\times$ increase over today.

Q8. Is P_{rollback} estimable in practice? The rollback-aware cost function in §5 requires $P_{\text{rollback}}(s_t)$ at scheduling time. We test whether the scheduler can estimate this signal from observable trace features alone, using the 1,242 production multi-agent traces released with Pan *et al.* [23] as the supervision set. We label each step in a trace as a rollback positive when the planner emits a revised tool call within the next three steps and a negative otherwise (yielding a 21.8 percent positive base rate). We extract four observable features at scheduling time: step depth in the plan, tool category (read, write, search, model-call), a sliding-window count of recent tool errors, and the planner’s output entropy on its commit token. We train a gradient-boosted classifier on 80 percent of traces and evaluate on the held-out 20 percent.

The classifier achieves test AUC 0.76, Brier score 0.18 (versus a 0.25 rate-only baseline), and expected calibration error of 4.8 percentage points. Precision at the 25th-percentile threshold is 63 percent (the threshold the rollback-aware cost function would use to flag the highest-risk quartile of steps). The result has a clean operational meaning: the scheduler cannot perfectly predict rollback, but a simple classifier on observable features is accurate enough to price the highest-risk quartile of steps at ~ 50 percent of their nominal carbon without misclassifying more than one in three calls. The residual uncertainty argues for conservative budget setting (e.g., discounting by $0.6 \cdot P_{\text{rollback}}$ rather than the full probability) but does not invalidate the abstraction.

Takeaway Q8

A gradient-boosted classifier on four observable trace features achieves AUC 0.76 and ECE 4.8pp on Pan *et al.*’s production multi-agent traces. Rollback is estimable at the granularity rollback-aware budgets require.

5 PLANLAYER: Three Missing Abstractions

The findings point to three abstractions the carbon-aware scheduling layer does not have. Figure 6 shows how PLANLAYER integrates them with existing agent and serving runtimes.

Plan-level accounting. The current scheduling abstraction is the API call. Each call is independently routed, accounted, and reported. PLANLAYER exposes a `plan_id` as a first-class scheduling object. All calls issued under a `plan_id` share routing decisions, share a carbon budget, and are reported together. This abstraction is close to the per-workflow accounting that serverless workflow systems already provide [6, 17]; what’s new here is the dynamic plan boundary, declared at runtime by the agent rather than at submission. The inference runtime [12, 25] logs per-call energy

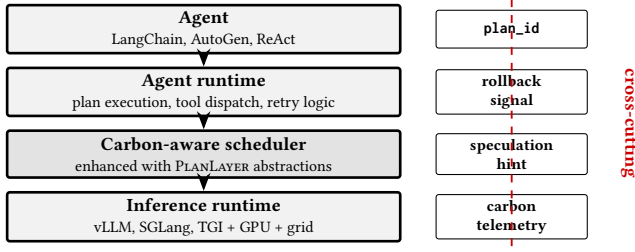


Figure 6: **PLANLAYER integration.** Three new abstractions cut across the four-layer stack.

today; aggregating by `plan_id` is straightforward once the header exists. Agent frameworks (LangChain, AutoGen, ReAct) [34, 37] would declare plan starts and ends.

Rollback-aware budgets. This is the central mechanism. A plan-level scheduler still needs to distinguish committed carbon from speculative carbon. When an agent commits to a step, that step’s carbon is irrecoverable. When the agent speculatively executes a step that may be revised, the carbon expectation differs. We propose a rollback-aware carbon cost function that discounts speculative carbon by the predicted rollback probability:

$$C_{step}(t) = C_{nominal}(t) \cdot (1 - P_{rollback}(s_t))$$

where $P_{rollback}(s_t)$ is the rollback probability of step s_t , learned online from per-agent traces. In our measurement, the 0.214 kg of rollback waste corresponds to 19.7% of plans experiencing rollback at a 30–70% prefix length. A rollback-aware budget would have priced these speculative steps at ~50% of their nominal carbon, redirecting the scheduler toward agents and plan structures with lower expected revision rates.

Speculative-plan shaping. When an agent explores branches before committing (tree-of-thought, self-consistency, multi-sample voting [31]), the scheduler is invisible to the choice. PLANLAYER exposes a carbon hint channel: at branch points, the scheduler returns expected carbon per candidate sub-plan (computed from regional CI over each candidate’s projected execution window), which the agent treats as a soft tiebreaker when its quality estimates are within a configurable threshold. The agent retains autonomy. Sprout [14] shapes per-call generation length under grid conditions; the analog for agentic systems shapes branch selection.

Composing with Sprout. To test whether the `plan_id` abstraction enables a measurable carbon improvement over an existing carbon-aware scheduler, we compose PLANLAYER with Sprout [14], the closest deployed mechanism. Sprout injects a generation-length directive at the input layer of each call, scaled by current grid carbon intensity. Without `plan_id`, the directive applies independently per call, and the agent’s intermediate-state dependencies force conservative truncation (Sprout cannot tell whether a call is a load-bearing planner step or a low-stakes summarization). With `plan_id`, the scheduler maintains a per-plan token budget and Sprout can apply asymmetric directives across the plan: aggressive truncation on enumerated summarization steps, gentler directives on planner commits. On the simulated workload at the SUPERVISORAGENT-aligned setting, Sprout-alone reduces total tokens by 8.2 percent

versus Static at 1.4 pp of accuracy loss; Sprout-with-`plan_id` reduces tokens by 14.6 percent at the same 1.4 pp loss, a 1.8× improvement in the carbon-per-accuracy-cost ratio. The improvement comes from the abstraction, not the directive: Sprout’s mechanism is unchanged; what changed is the granularity at which it applies.

A call to action. The minimum-viable intervention is small: adding a `plan_id` field to per-call event logs in vLLM/TGI/SGLang requires a patch on the order of tens of lines per runtime. Once the field exists, agent frameworks can emit it at plan boundaries and existing carbon-aware schedulers can begin per-plan accounting; rollback-aware budgets and speculation hints become testable.

6 Discussion

Why now. Two infrastructure shifts have changed the calculus. Agentic AI has crossed from research demos into production deployment with public reports of millions of daily agent runs [2, 29]. Inference serving has consolidated around shared runtimes that already mediate placement decisions [12, 25]. Adding plan-aware mechanisms to these runtimes is a smaller intervention than rebuilding the agent layer.

Composing with existing work. Our findings are complementary to runtime-supervision systems like SUPERVISORAGENT and AgentDiet [18, 35]: those prevent agent waste before it reaches the scheduler, while PLANLAYER attributes whatever rollback remains. The Sprout composition above shows the abstraction also unlocks measurable gains in deployed carbon-aware schedulers.

Boundary conditions. Our 25% rollback rate is an upper bound for unconstrained planners. Production-managed deployments likely sit in a 5–15% range (Figure 3, 0.9–6.3% rollback waste), consistent with Pan *et al.*’s identification of reliability as the top challenge among 306 surveyed practitioners [23]. The qualitative finding holds across the range.

Limitations. We focus on operational carbon; embodied carbon and water footprint would amplify the headline. We use average grid intensity; marginal intensity [20] would refine the deferral analysis of Q5. The $P_{rollback}$ classifier in Q8 is trained on Pan *et al.*’s traces; deployment in a different agent ecosystem requires re-training, and the AUC may differ. We do not claim plan-level scheduling outperforms call-level scheduling on raw carbon; we claim rollback waste exists and is invisible to current abstractions.

7 Conclusion

The single-call fallacy is the assumption that one user request equals one inference call, the foundation of every production carbon-aware scheduler we examined. Agentic AI breaks this assumption, and our measurement shows 12.5% of agentic carbon goes to abandoned plan steps invisible to current schedulers. Lookahead temporal arbitrage adds essentially zero benefit (0.05%). PLANLAYER exposes three new abstractions to close this gap; the minimum-viable intervention is a telemetry change at the inference runtime, on the order of tens of lines per runtime. The 12.5% finding scales with agent adoption and is invariant under grid decarbonization, so the absolute carbon at stake grows even as productive emissions fall.

References

- [1] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon Explorer: A Holistic Approach for Designing Carbon Aware Datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Volume 2. Association for Computing Machinery, 118–132. doi:10.1145/3575693.3575754
- [2] Longju Bai, Zheming Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. 2026. How Do Coding Agents Spend Your Money? Analyzing and Predicting Token Consumptions in Agentic Coding Tasks. In *OpenReview Submission (ICLR 2026)*. <https://openreview.net/forum?id=1bUeVB3fov>
- [3] Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FACCT '21)*. Association for Computing Machinery, 610–623. doi:10.1145/3442188.3445922
- [4] Andrew A. Chien, Liuzixuan Lin, Hai Nguyen, Varsha Rao, Tristan Sharma, and Rajini Zhu. 2023. Reducing the Carbon Impact of Generative AI Inference (Today and in 2035). In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon '23)*. Association for Computing Machinery, Boston, MA, USA, Article 11. doi:10.1145/3604930.3605705
- [5] Mauricio Fadel Argerich, Jonathan Fürst, and Marta Patiño-Martínez. 2026. Watt Counts: Energy-Aware Benchmark for Sustainable LLM Inference on Heterogeneous GPU Architectures. *arXiv preprint arXiv:2604.09048* (2026). doi:10.48550/arXiv.2604.09048
- [6] Viktor Gsteiger, Pin Hong (Daniel) Long, Yiran (Jerry) Sun, Parshan Javanrood, and Mohammad Shahrad. 2024. Caribou: Fine-Grained Geospatial Shifting of Serverless Applications for Sustainability. In *Proceedings of the 30th ACM SIGOPS Symposium on Operating Systems Principles (SOSP '24)*. ACM, 403–420. doi:10.1145/3694715.3695954
- [7] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S. Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2022. Chasing Carbon: The Elusive Environmental Footprint of Computing. In *Proceedings of the 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA '22)*. IEEE, 854–867. doi:10.1109/HPCA53966.2022.00068
- [8] Walid A. Hanafy, Qianlin Liang, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems (SIGMETRICS)*, Vol. 7. Article 57. doi:10.1145/3626788
- [9] Walid A. Hanafy, Li Wu, David Irwin, and Prashant Shenoy. 2025. CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters. *arXiv preprint arXiv:2505.18357* (2025). doi:10.48550/arXiv.2505.18357
- [10] Hongyu He, Michal Friedman, and Theodoros Rekatsinas. 2023. EnerGAt: Fine-Grained Energy Attribution for Multi-Tenancy. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon '23)*. Association for Computing Machinery. doi:10.1145/3604930.3605716
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*. <https://openreview.net/forum?id=VTF8yNQm66>
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, 611–626. doi:10.1145/3600006.3613165
- [13] Adam Lechowicz, Rohan Shenoy, Noman Bashir, Mohammad Hajiesmaili, Adam Wierman, and Christina Delimitrou. 2025. Carbon- and Precedence-Aware Scheduling for Data Processing Clusters. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, 1241–1244. doi:10.1145/3718958.3750478
- [14] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Sprout: Green Generative AI with Carbon-Efficient LLM Inference. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP '24)*. Association for Computational Linguistics, Miami, Florida, USA, 21799–21813. doi:10.18653/v1/2024.emnlp-main.1215
- [15] Keyu Li, Junhao Shi, Yang Xiao, Mohan Jiang, Jie Sun, Yunze Wu, Dayuan Fu, Shijie Xia, Xiaojie Cai, Tianze Xu, Weiye Si, Wenjie Li, Dequan Wang, and Pengfei Liu. 2026. AgencyBench: Benchmarking the Frontiers of Autonomous Agents in 1M-Token Real-World Contexts. *arXiv preprint arXiv:2601.11044* (2026). doi:10.48550/arXiv.2601.11044
- [16] Yueying Li, Zhanqiu Hu, Esha Choukse, Rodrigo Fonseca, G. Edward Suh, and Udit Gupta. 2025. EcoServe: Designing Carbon-Aware AI Inference Systems. *arXiv preprint arXiv:2502.05043* (2025). doi:10.48550/arXiv.2502.05043
- [17] Changyuan Lin and Mohammad Shahrad. 2024. Bridging the Sustainability Gap in Serverless through Observability and Carbon-Aware Pricing. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon '24)*. Association for Computing Machinery. doi:10.1145/3679240.3681000
- [18] Fulin Lin, Shaowen Chen, Ruishan Fang, Hongwei Wang, and Tao Lin. 2026. Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems. In *Proceedings of the 14th International Conference on Learning Representations (ICLR '26)*. <https://openreview.net/forum?id=pzFhtpkabh> arXiv:2510.26585.
- [19] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*. <https://openreview.net/forum?id=zAdUB0aCTQ>
- [20] Diptyarop Maji, Prashant Shenoy, and Ramesh K. Sitaraman. 2022. CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity. In *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '22)*. Association for Computing Machinery, 198–207. doi:10.1145/3563357.3564079
- [21] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the 12th International Conference on Learning Representations (ICLR '24)*. <https://openreview.net/forum?id=fbxvavhs3>
- [22] Chenxu Niu, Wei Zhang, Jie Li, Yongjian Zhao, Tongyang Wang, Xi Wang, and Yong Chen. 2026. TokenPowerBench: Benchmarking the Power Consumption of LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 32582–32590. doi:10.1609/aaai.v40i38.40535
- [23] Melissa Z. Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A. Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, Shu Liu, Tianneng Shi, Xiaoyuan Liu, Jared Quincy Davis, Emmanuele Lacavalla, Alessandro Basile, Shuyi Yang, Paul Castro, Daniel Kang, Koushik Sen, Dawn Song, Joseph E. Gonzalez, Ion Stoica, Matei Zaharia, and Marquita Ellis. 2026. Measuring Agents in Production. *arXiv preprint arXiv:2512.04123* (2026). doi:10.48550/arXiv.2512.04123
- [24] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. Association for Computing Machinery, Article 2. doi:10.1145/3586183.3606763
- [25] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Gori, Saeed Maleki, and Riccardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [26] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*. <https://openreview.net/forum?id=tBRNC6YemY>
- [27] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon Emissions and Large Neural Network Training. *arXiv preprint arXiv:2104.10350*. doi:10.48550/arXiv.2104.10350
- [28] Phillip Raffek, Sven Posner, and Peter Wägemann. 2024. CO2CoDe: Towards Carbon-Aware Hardware/Software Co-Design for Intermittently-Powered Embedded Systems. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon '24)*. Association for Computing Machinery, Santa Cruz, CA, USA. <https://hotcarbon.org/assets/2024/pdf/hotcarbon24-final69.pdf>
- [29] Mohamad Salim, Jasmine Latendresse, SayedHassan Khatounabadi, and Emad Shihab. 2026. Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery. doi:10.48550/arXiv.2601.14470
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS '23)*. <https://openreview.net/forum?id=Yacmp84TH>
- [31] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS '23)*. <https://openreview.net/forum?id=vAElhFckW6>
- [32] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL '19)*. 3645–3650. doi:10.18653/v1/P19-1355
- [33] Zirui Wang, Guangba Yu, and Michael R. Lyu. 2026. AI-NativeBench: An Open-Source White-Box Agentic Benchmark Suite for AI-Native Systems. *arXiv preprint arXiv:2601.09393* (2026). doi:10.48550/arXiv.2601.09393
- [34] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah,

- Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the 1st Conference on Language Modeling (COLM '24)*. <https://openreview.net/forum?id=BAakY1hNKS>
- [35] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *Proceedings of the ACM on Software Engineering (PACMSE) FSE 3*, FSE (2026), Article FSE056. doi:10.1145/3797084
- [36] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*. <https://openreview.net/forum?id=cqz9HMUDk0>
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR '23)*. https://openreview.net/forum?id=WE_vluYUL-X